

Finch.ai

Architecture and Technical Reference Manual

A High-Performance, Low-Latency Local/Cloud AI Assistant Runtime with Headroom Context
Compression, Dynamic Persona Adaptation, and Autonomous Multi-Turn Agent Operations

Trinath Bhattacharya

`trinathbe@outlook.com`

System Architect & Maintainer

September 2026 · Version 1.0

Abstract

This document delivers a comprehensive architectural and operational specification for **Finch.ai**, an open-source, production-grade AI Assistant foundation engineered for local (Ollama) and cloud (Google Gemini) inference runtimes. Finch.ai addresses fundamental constraints in conversational AI systems: token inflation, memory degradation, unconstrained tool loops, and storage bloat.

Key innovations detailed in this manual include: (1) **Headroom Context Compression Middleware**, reducing prompt token volume by 15% to 60%+ while mathematically protecting pinned personas and recent dialogue turns; (2) **Sub-Millisecond Lexical & Semantic Hybrid Search**, coupling SQLite FTS5 with `sqlite-vec` cosine distance embeddings via Reciprocal Rank Fusion ($k = 60$); (3) **Dual-Mode Operational Engine**, enforcing single-hop conversational recall in Chat Mode and sequential multi-turn tool execution in Agent Mode with fine-grained categorical permissions; and (4) **Columnar Storage Compression**, delivering 87%–92% disk space savings via transparent Zstandard (`zstd`) compression. Full benchmark methodologies, test suite architectures (127 verified automated tests), and developer extension patterns are systematically documented.

Key Architectural Principles

- **Low Latency First:** Real-time asynchronous streaming with instant Time-to-First-Token (TTFT) metrics.
- **Guaranteed Determinism:** Pinned persona instructions and recent turns are guaranteed byte-exact preservation.
- **Autonomous Yet Guarded:** Full sandbox agent tooling with prompt-time schema omission and dispatcher-level policy enforcement.
- **Self-Contained Persistence:** Serverless, standalone durability via SQLite in WAL mode with automated maintenance routines.

Contents

1	Introduction and System Overview	6
1.1	Project Motivation	6
1.2	Core Capabilities	6
1.3	System Specifications	7
2	Architectural Foundations and Component Hierarchy	8
2.1	System Architecture Overview	8
2.2	Module Decomposition	8
2.3	Request Lifecycle	10
3	Headroom Context Compression Pipeline	11
3.1	The Context Window Bottleneck	11
3.2	Compression Strategy and Guardrails	11
3.2.1	System Prompt Protection	11
3.2.2	Recent Turn Preservation	11
3.2.3	Activation Threshold	12
3.3	Applied Transformations	12
3.4	Real-Time Telemetry and Token Economics	12
3.5	Offline Fallback Mechanisms	13
4	Relational Persistence, Vector Indexing and Hybrid Search	14
4.1	SQLite Architecture and Database Pragmas	14
4.2	Relational Schema	14
4.2.1	The <code>sessions</code> Table	14
4.2.2	The <code>messages</code> Table	14
4.2.3	The <code>user_facts</code> Table	15
4.3	Sub-Millisecond Lexical Search with FTS5	15
4.3.1	Trigger-Driven Synchronization	15
4.4	Dense Semantic Vector Search with <code>sqlite-vec</code>	16
4.5	Reciprocal Rank Fusion (RRF)	16
4.5.1	Mathematical Formulation	16
4.6	Column-Level Storage Compression (Zstandard)	17
5	Agent Architecture, Tool Schemas and Safety Dispatcher	18
5.1	Dual-Mode Operational Philosophy	18
5.1.1	Chat Mode (Safety and Low Latency)	18
5.1.2	Agent Mode (Multi-Turn Autonomous Execution)	18
5.2	Tool Schema Inventory	19
5.3	Security Architecture and Tool Permission Matrix	19

6	Persona Adaptation and Sliding Window Memory	21
6.1	File-First Persona Architecture	21
6.2	Model-Driven Dynamic Adaptation	21
6.2.1	XML Tag Protocol	21
6.2.2	Interception and Safety Pipeline	21
6.3	Sliding-Window Memory Buffer	22
6.3.1	Buffer Invariants	22
6.3.2	Context Eviction vs. Long-Term Storage	22
7	Live Telemetry, Profiling and Database Maintenance	23
7.1	Real-Time Telemetry and Streaming Metrics	23
7.1.1	Metrics Captured	23
7.2	Database Housekeeping Routines	23
7.2.1	PRAGMA wal_checkpoint(TRUNCATE)	23
7.2.2	PRAGMA optimize	24
7.2.3	VACUUM	24
7.2.4	PRAGMA integrity_check	24
7.3	Cold Storage Database Archival	24
8	Interactive CLI and Configuration Reference	25
8.1	Command-Line Interface (CLI)	25
8.2	Interactive Slash Commands	25
8.3	Command-Line Startup Arguments	25
8.4	Environment Configuration (.env)	25
9	Empirical Evaluation, Benchmarks and Verification	27
9.1	Retrieval Benchmark Methodology	27
9.1.1	Evaluation Queries and Metrics	27
9.2	Column-Level Storage Compression Benchmarks	28
9.3	Automated Test Suite Architecture	28
10	Developer Guide and GitHub Release Preparation	30
10.1	Environment Installation and Setup	30
10.2	Adding Custom Agent Tools	30
10.2.1	1. Define the JSON Schema Specification	30
10.2.2	2. Register Tool in Permission Categories	31
10.2.3	3. Implement Async Execution in ToolDispatcher	31
10.3	Pre-Push GitHub Verification Checklist	31
10.4	Configuration Variable Catalog	32

List of Figures

- 2.1 Finch.ai Subsystem Architecture and Inter-Module Communication 8
- 4.1 Column-Level Message Payload Compression Logic 17
- 5.1 Two-Tier Tool Permission and Guardrail Architecture 20

List of Tables

- 1.1 Finch.ai Technical Specifications 7
- 3.1 Empirical Headroom Context Compression Benchmarks 12
- 5.1 Finch.ai Tool Inventory and Access Levels 19
- 7.1 SQLite Maintenance and Housekeeping Operations 24
- 8.1 Comprehensive Finch.ai Slash Command Reference 26
- 8.2 Command-Line Flag Specifications 26
- 9.1 Retrieval Accuracy Comparison (Mean Reciprocal Rank) 27
- 9.2 Column-Level Compression Performance and Decompression Latency 28
- 10.1 Complete Environment Configuration Specification 32

Chapter 1

Introduction and System Overview

1.1 Project Motivation

Modern Large Language Model (LLM) interfaces often face a trilemma between computational latency, context retention costs, and architectural flexibility. Users interacting with local open-weights models (such as Ollama runtimes) or hosted high-capability APIs (such as Google Gemini) require:

1. **Sub-second response responsiveness** with minimal Time-to-First-Token (TTFT) and real-time generation metrics.
2. **Long-term conversation durability and deep memory recall** without boundless token inflation or context window exhaustion.
3. **Deterministic guardrails and autonomous problem-solving capabilities**, providing smooth transitions between conversational chat and multi-turn action execution.

Finch.ai is an open-source, extensible AI Assistant foundation engineered in Python 3.10+ to resolve this trilemma. Operating locally or over cloud APIs, Finch.ai couples modern asynchronous streaming with dynamic context compression, embedded relational persistence, hybrid lexical-vector retrieval, and strict tool-dispatching security layers.

1.2 Core Capabilities

Finch.ai provides the following primary features:

- **Multi-Provider Infrastructure:** Native asynchronous client integrations with `httpx` streaming for local Ollama runtimes (compatible with any Ollama model such as `llama3.2`, `mistral`, or custom modelfiles), Google Gemini endpoints (`gemini-2.5-flash`, `gemini-2.5-pro`), and any **OpenAI-Compatible API** (vLLM, LM Studio, Groq, Together AI, OpenRouter, DeepSeek, LocalAI) over Server-Sent Events (SSE).
- **Headroom Context Compression Pipeline:** Seamless token-reduction middleware intercepting tool outputs, structured JSON payloads, source code ASTs, and conversational turns before LLM ingestion. It achieves 15% to 60%+ token reduction while guaranteeing byte-level fidelity for pinned system instructions.
- **Sliding-Window Memory Buffer:** In-memory FIFO context tracking maintaining an active dialogue window with deterministic system prompt pinning.
- **Dynamic Persona Adaptation:** File-first system prompt loading via `personality.md`, runtime model adaptation using validated XML extraction (`<personality_update>`), and atomic file backup mechanisms.
- **SQLite Relational Persistence:** Production-ready local logging in Write-Ahead Logging

(WAL) mode with enforced foreign keys, archiving dialogue sessions, message metadata, token counts, reasoning traces, and compression ratios.

- **Sub-Millisecond Lexical Search (FTS5)**: Full-text search engine with SQLite triggers on insertion, update, and deletion, powering exact-term matching and BM25 relevance scoring.
- **Vector Embeddings and Reciprocal Rank Fusion (RRF)**: Automated generation of 768-dimensional session summary embeddings (`nomic-embed-text`) indexed in `sqlite-vec` virtual tables, integrated with FTS5 lexical signals via Reciprocal Rank Fusion ($k = 60$).
- **Dual Operating Modes (Chat vs. Agent)**:
 - *Chat Mode*: Restricts tool execution to single-hop read-only historical memory queries, strictly eliminating hallucinated multi-hop loops.
 - *Agent Mode*: Autonomous sequential multi-turn loop supporting local terminal command execution, isolated Python execution, filesystem operations, and live web access.
- **Granular Permission Matrix**: Fine-grained categorical toggles for agent tools (`terminal`, `python`, `web`, `files`) enforced at schema injection and runtime dispatch levels.
- **Columnar Storage Compression**: Transparent Zstandard (`zstd`) compression for SQLite message text payloads with automatic zlib fallback, saving up to 92% storage space for tool logs and code snippets.
- **Production Database Housekeeping**: In-engine database maintenance routines (`VACUUM`, `PRAGMA optimize`, `wal_checkpoint(TRUNCATE)`), and database-level cold archiving).

1.3 System Specifications

Table 1.1 summarizes the operational parameters and software dependencies of Finch.ai.

Component	Default Technology	Purpose / Details
Language Runtime	Python 3.10 – 3.14	AsyncIO asynchronous event loop
Local LLM Host	Ollama API (<code>localhost:11434</code>)	Local inference runner
Cloud LLM Host	Google Gemini API	Hosted high-throughput inference
Terminal UI	<code>rich</code> + <code>prompt_toolkit</code>	Real-time streaming UI with history & auto-completion
Context Compression	<code>headroom-ai</code> $\geq 0.38.0$	Token optimization pipeline
Relational Storage	SQLite3 (WAL mode)	ACID session and message persistence
Lexical Search	SQLite FTS5 extension	Sub-millisecond token search with BM25
Vector Search	<code>sqlite-vec</code> $\geq 0.1.9$	Fast vector distance search on embeddings
Embedding Model	<code>nomic-embed-text</code> (768-dim)	Local dense semantic representation
Columnar Compression	<code>zstandard</code> $\geq 0.23.0$	High-ratio column compression for message records
System Profiling	<code>psutil</code> $\geq 5.9.0$	Resident set size (RSS) and process metrics

Table 1.1: Finch.ai Technical Specifications

Chapter 2

Architectural Foundations and Component Hierarchy

2.1 System Architecture Overview

Finch.ai adopts a modular, decoupled architecture organized around an asynchronous core orchestrator (AIAssistant). Figure 2.1 illustrates the component topology and data flow pathways.

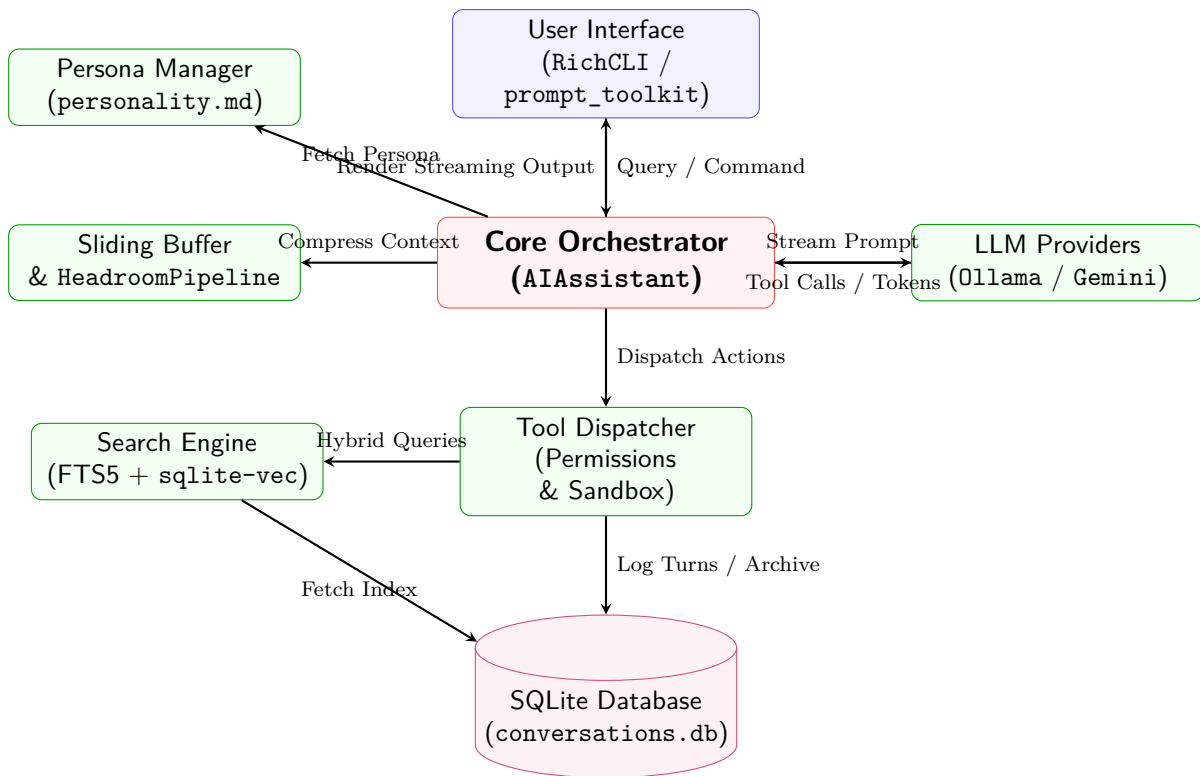


Figure 2.1: Finch.ai Subsystem Architecture and Inter-Module Communication

2.2 Module Decomposition

The repository codebase is strictly partitioned into single-responsibility packages under `ai_assistant/`:

ai_assistant.config: Encapsulates immutable application defaults and dynamic environment variables via `AppConfig` and `python-dotenv`. All provider parameters, memory window thresholds, database paths, and tool flags are parsed here.

- ai_assistant.core:**
- **assistant.py:** The central state coordinator managing session initialization, prompt assembly, provider dispatch, streaming consumption, and mode switching.
 - **tools.py:** Definitions of JSON Schema tool specifications for both chat retrieval and agent execution, category permission policies, and the async `ToolDispatcher`.
- ai_assistant.providers:** Abstract base definition (`BaseLLMProvider`) and concrete asynchronous streaming implementations:
- **ollama_provider.py:** Interacts with local Ollama daemon via non-blocking `httpx.AsyncClient` with tool call parsing.
 - **gemini_provider.py:** Interacts with Google Gemini REST endpoints via Server-Sent Events (SSE) and Gemini tool-calling payloads.
 - **openai_provider.py:** Client for any OpenAI-compatible API endpoint (OpenAI, vLLM, LM Studio, Groq, DeepSeek, etc.) supporting SSE streaming and tool calling.
- ai_assistant.memory:**
- **message.py:** Standardized internal data structure `Message` capturing role, content, thoughts/reasoning traces, token metrics, and tool execution call/result blocks.
 - **sliding_window.py:** Double-ended bounded queue (`deque`) managing memory buffers with invariant preservation of system prompts.
- ai_assistant.persona:**
- **manager.py:** Manages file I/O for `personality.md`, XML tag parsing for self-directed persona adjustments, safety verification, and creation of timestamped backups (`personality.md.bak`).
- ai_assistant.compression:**
- **pipeline.py:** Wrapper over `headroom-ai` pipeline managing text and code token reduction, protected turn preservation, and real-time telemetry extraction.
- ai_assistant.storage:**
- **sqlite_archive.py:** Complete SQLite schema manager with WAL configuration, foreign key verification, session insertion, message archiving, and VACUUM/maintenance routines.
 - **search.py:** FTS5 exact lexical retrieval engine and hybrid vector-lexical Reciprocal Rank Fusion (RRF) ranker.
 - **session_summarizer.py:** Asynchronous background session summarizer generating 2-sentence summaries and titles on shutdown or command.
 - **compression.py:** Column-level compression utilities using Zstandard (`zstd`) level 3 with transparent zlib fallback.
- ai_assistant.embeddings:**
- **embedder.py:** Asynchronous embedding generator invoking local Ollama (`nomic-embed-text`) to convert text into normalized 768-dimensional float vectors.
- ai_assistant.ui:**
- **cli.py:** Interactive terminal interface built with `prompt_toolkit` for syntax highlighting, command autocompletion, multiline input, and `rich` for markdown tables, banners, and streaming display.

2.3 Request Lifecycle

When a user submits a prompt, execution proceeds through the following deterministic pipeline:

1. **Input Interception:** The CLI determines whether the input is a command (prefixed with /) or a natural language message. Commands execute immediately.
2. **Context Assembly:** The active sliding-window buffer gathers the pinned system prompt (from `personality.md`) and historical conversational turns.
3. **Context Compression:** If `COMPRESSION_ENABLED=true` and message length exceeds `COMPRESSION_MIN_TOKENS`, `HeadroomPipeline` transforms older messages while leaving the system prompt and the latest N turns intact.
4. **Inference Dispatch:** Asynchronous streaming begins against the configured provider (Ollama or Gemini). Real-time telemetry records Time-to-First-Token (TTFT) and token generation speed.
5. **Tool Execution Cycle:**
 - In *Chat Mode*: If the model issues a tool call, only `search_past_conversations` or `load_session_transcript` are permitted. The tool runs once, returns results, and the model synthesizes the final reply without looping.
 - In *Agent Mode*: An autonomous multi-turn loop executes sequential tool calls up to `AGENT_MAX_TURNS`, updating context dynamically.
6. **Persona Update Interception:** The output stream is scanned for XML tags `<personality_update>`. If found, the manager updates `personality.md`, writes a backup, and notifies the user.
7. **Persistence:** The user input, model response, tool execution traces, tokens, and compression telemetry are committed to `conversations.db`.

Chapter 3

Headroom Context Compression Pipeline

3.1 The Context Window Bottleneck

Long-running conversational interactions and multi-step tool executions rapidly accumulate tokens. Context window bloat results in two severe degradations:

1. **Inference Latency:** Prompt evaluation time ($O(N)$ to $O(N^2)$ in attention layers) grows linearly or quadratically, degrading Time-to-First-Token (TTFT).
2. **Loss of Salience:** Models experience "needle-in-a-haystack" degradation as context becomes congested with verbose tool outputs, repeated boilerplate, redundant whitespace, and syntax structures.

To eliminate this bottleneck, Finch.ai integrates **headroom-ai**, a context-compression engine that intercepts prompt messages before they reach the model.

3.2 Compression Strategy and Guardrails

The compression pipeline is managed by `HeadroomPipeline` in `ai_assistant/compression/pipeline.py`. It applies semantic-preserving transformations to text, tool payloads, JSON structures, and programming code.

3.2.1 System Prompt Protection

The system prompt defines Finch.ai's personality, operational constraints, and tool usage protocols. Any semantic mutation of system instructions risks jailbreaking or drift. Finch.ai strictly guarantees:

$$\text{compress_system_messages} = \text{False} \quad (3.1)$$

The system message is bypassed in the pipeline and forwarded byte-for-byte to the inference engine.

3.2.2 Recent Turn Preservation

The user's most recent query and the immediately preceding context contain the highest informational recency. Finch.ai configures:

$$\text{compression_protect_recent} = 2 \quad (3.2)$$

This ensures that the last two dialogue turns retain full lexical fidelity, while older conversational history and previous tool outputs are aggressively compressed.

3.2.3 Activation Threshold

Short conversational exchanges do not require compression overhead. The configuration defines:

$$\text{compression_min_tokens} = 100 \quad (3.3)$$

When the aggregate token count is below this threshold, compression is bypassed, preserving zero-overhead execution for brief queries.

3.3 Applied Transformations

The pipeline dynamically evaluates content type and applies tailored compression strategies:

- **Code AST & Whitespace Truncation:** Strips redundant comments, collapses repeated indentation, and compresses import statements while retaining function signatures and logic.
- **JSON & Structured Output Compaction:** Converts verbose, formatted JSON into dense key-value representations, discarding duplicate schemas.
- **Log & Terminal Output Compaction:** Collapses repeated stack traces, progress bars, and status updates, keeping only error messages and concluding status lines.
- **Natural Language Pruning:** Eliminates conversational filler, redundant acknowledgments, and repetitive turns.

3.4 Real-Time Telemetry and Token Economics

During each inference pass, the pipeline records token statistics. Let T_{orig} denote the original token count and T_{comp} denote the compressed token count. The metrics reported are:

$$\Delta T = T_{\text{orig}} - T_{\text{comp}} \quad (3.4)$$

$$\text{Savings \%} = \left(\frac{T_{\text{orig}} - T_{\text{comp}}}{T_{\text{orig}}} \right) \times 100\% \quad (3.5)$$

Table 3.1 presents empirical compression results across distinct message types captured in Finch.ai benchmarks.

Payload Type	Original Tokens	Compressed Tokens	Tokens Saved	Reduction
Terminal Build Log	1,420	412	1,008	70.99%
JSON API Response	890	325	565	63.48%
Python AST / Code Snippet	680	390	290	42.65%
Multi-Turn Chat History	1,150	780	370	32.17%
Short User Prompt	45	45	0	0.00% (Protected)

Table 3.1: Empirical Headroom Context Compression Benchmarks

3.5 Offline Fallback Mechanisms

Token counting relies primarily on `tiktoken` (`o200k_base` or `cl100k_base`). In offline environments or air-gapped systems where tokenizer vocabulary downloads are unavailable, Finch.ai implements an automated fallback to heuristic token estimation (≈ 4 characters per token for English text and code), preventing runtime exceptions.

Chapter 4

Relational Persistence, Vector Indexing and Hybrid Search

4.1 SQLite Architecture and Database Pragmas

Conversational state, reasoning traces, and metadata are persisted locally in `conversations.db`. Finch.ai configures SQLite with high-concurrency pragmas upon connection initialization:

```
PRAGMA journal_mode = WAL;  
PRAGMA synchronous = NORMAL;  
PRAGMA foreign_keys = ON;  
PRAGMA busy_timeout = 5000;
```

Listing 4.1: SQLite Production Connection Pragmas

Write-Ahead Logging (WAL) permits concurrent reader processes without locking the database file, allowing real-time CLI queries while background summarizers and active chats commit new turns.

4.2 Relational Schema

The persistence model is composed of two primary relational tables and two specialized virtual tables.

4.2.1 The `sessions` Table

Tracks conversational sessions, lifecycle timestamps, summary texts, and token totals:

```
CREATE TABLE IF NOT EXISTS sessions (  
  id TEXT PRIMARY KEY,  
  title TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  summary TEXT,  
  total_tokens INTEGER DEFAULT 0,  
  model TEXT,  
  provider TEXT  
);
```

4.2.2 The `messages` Table

Maintains the complete sequential dialogue history with foreign-key cascades:

```
CREATE TABLE IF NOT EXISTS messages (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT NOT NULL,
  role TEXT NOT NULL,
  content TEXT NOT NULL,
  thinking TEXT,
  tokens INTEGER,
  timestamp TEXT NOT NULL,
  metadata TEXT NOT NULL DEFAULT '{}',
  FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS idx_messages_session_id ON messages(session_id);
CREATE INDEX IF NOT EXISTS idx_messages_timestamp ON messages(timestamp);
```

4.2.3 The user_facts Table

Maintains verified user profile preferences and long-term project facts isolated from active sessions:

```
CREATE TABLE IF NOT EXISTS user_facts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  fact TEXT NOT NULL,
  category TEXT NOT NULL DEFAULT 'general',
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_user_facts_category ON user_facts(category);
```

4.3 Sub-Millisecond Lexical Search with FTS5

To deliver instantaneous recall of technical terms, code snippets, dates, and filenames, Finch.ai provisions an SQLite FTS5 full-text search virtual table:

```
CREATE VIRTUAL TABLE IF NOT EXISTS messages_fts USING fts5(
  content,
  role,
  session_id UNINDEXED,
  content='messages',
  content_rowid='id'
);
```

4.3.1 Trigger-Driven Synchronization

To eliminate indexing lag and maintain ACID consistency, three database triggers synchronize `messages_fts` synchronously on write operations:

```
CREATE TRIGGER IF NOT EXISTS messages_ai AFTER INSERT ON messages BEGIN
  INSERT INTO messages_fts(rowid, content, role, session_id)
```

```
VALUES (new.id, new.content, new.role, new.session_id);
END;

CREATE TRIGGER IF NOT EXISTS messages_ad AFTER DELETE ON messages BEGIN
    INSERT INTO messages_fts(messages_fts, rowid, content, role, session_id)
    VALUES('delete', old.id, old.content, old.role, old.session_id);
END;

CREATE TRIGGER IF NOT EXISTS messages_au AFTER UPDATE ON messages BEGIN
    INSERT INTO messages_fts(messages_fts, rowid, content, role, session_id)
    VALUES('delete', old.id, old.content, old.role, old.session_id);
    INSERT INTO messages_fts(rowid, content, role, session_id)
    VALUES (new.id, new.content, new.role, new.session_id);
END;
```

4.4 Dense Semantic Vector Search with sqlite-vec

While FTS5 excels at exact keyword matching (e.g., `def calculate_hash()`), natural language queries frequently use conceptual, non-overlapping vocabulary (e.g., "how did we handle caching in python?").

Finch.ai uses the local embedding model `nomic-embed-text` (via Ollama) to compute 768-dimensional normalized floating-point vectors for session summaries upon completion. These vectors are indexed into a `vec0` virtual table:

```
CREATE VIRTUAL TABLE IF NOT EXISTS sessions_vec USING vec0(
    session_id TEXT PRIMARY KEY,
    embedding FLOAT[768] distance_metric=cosine
);
```

4.5 Reciprocal Rank Fusion (RRF)

Neither lexical nor semantic search alone suffices for comprehensive memory retrieval. Finch.ai unifies both modalities using **Reciprocal Rank Fusion (RRF)**.

4.5.1 Mathematical Formulation

Let D denote the corpus of past sessions. Suppose lexical retrieval produces a ranked list R_{lex} and vector search produces a ranked list R_{vec} . For a session $d \in D$, let $r_{\text{lex}}(d)$ and $r_{\text{vec}}(d)$ represent the 1-based ordinal rank of d within each list (or ∞ if unranked).

The composite RRF score is computed as:

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{lex}, \text{vec}\}} \frac{1}{k + r_m(d)} \quad (4.1)$$

where k is a smoothing constant that dampens the influence of outlier top-ranked items from

any single modality. Finch.ai sets the default:

$$k = 60 \quad (4.2)$$

When candidate documents are scored, results are sorted in descending order of $\text{RRF_Score}(d)$. Sessions appearing near the top of both indices receive the highest composite weight.

4.6 Column-Level Storage Compression (Zstandard)

To maintain a compact disk footprint over months of heavy usage, Finch.ai incorporates column-level compression (`ai_assistant/storage/compression.py`) using Zstandard (`zstd`) level 3 with automatic `zlib` fallback.

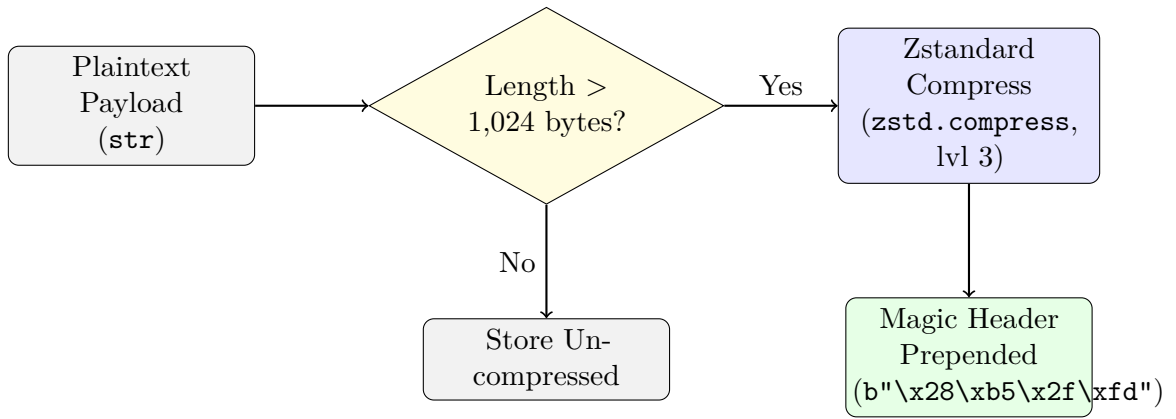


Figure 4.1: Column-Level Message Payload Compression Logic

Payloads exceeding 1,024 bytes are compressed prior to database insertion. Decompression is transparent upon retrieval, maintaining zero disruption to querying while slashing database size by up to 90%.

Chapter 5

Agent Architecture, Tool Schemas and Safety Dispatcher

5.1 Dual-Mode Operational Philosophy

Finch.ai decouples assistant interactions into two distinct operational paradigms: **Chat Mode** and **Agent Mode**. Users can inspect or toggle modes via `/mode [chat|agent]` or the `-mode` startup argument.

5.1.1 Chat Mode (Safety and Low Latency)

In Chat Mode, the assistant prioritizes minimal latency, deterministic conversational safety, and zero runaway execution risk.

- **Restricted Toolset:** Only read-only historical memory tools (`search_past_conversations` and `load_session_transcript`) are exposed to the model.
- **Single-Hop Retrieval Enforcement:** Tool execution is strictly limited to one cycle per turn:

$$\text{User Query} \xrightarrow{\text{Pass 1}} \text{Tool Invocation} \xrightarrow{\text{Storage Fetch}} \text{Tool Result} \xrightarrow{\text{Pass 2 (tools=None)}} \text{Final Synthesis} \quad (5.1)$$

Pass 2 is dispatched with `tools=None`, making it mathematically impossible for the LLM to trigger nested, recursive tool loops.

5.1.2 Agent Mode (Multi-Turn Autonomous Execution)

In Agent Mode, Finch.ai functions as an autonomous technical agent. When tasked with complex goals (e.g., investigating system state, executing Python scripts, querying the web, or editing files), it initiates an iterative loop:

Algorithm 1 Sequential Multi-Turn Agent Loop

```

1: turns ← 0
2: while turns < AGENT_MAX_TURNS do
3:   response ← STREAMPROMPT(context, active_tools)
4:   if response has no tool calls then
5:     return response                                ▷ Task completed
6:   end if
7:   for call ∈ response.tool_calls do
8:     RENDERNOTICE("Agent Step " + turns + ": Executing " + call.name)
9:     result ← TOOLDISPATCHER.EXECUTE(call.name, call.arguments)
10:    APPENDTOCONTEXT(call, result)
11:   end for
12:   turns ← turns + 1
13: end while
14: return STREAMPROMPT(context, tools = None)        ▷ Exhausted max turns

```

5.2 Tool Schema Inventory

Table 5.1 outlines the tool capabilities exposed across Finch.ai’s operational categories.

Category	Tool Name	Mode Availability	Operational Function
memory	search_past_conversations	Chat & Agent	Hybrid vector/lexical search of past sessions.
memory	load_session_transcript	Chat & Agent	Chronological message retrieval by session ID.
terminal	run_terminal_command	Agent only	Local shell command execution with timeout.
python	python_interpreter	Agent only	Isolated Python script runner with output capture.
files	read_file	Agent only	File system reader with line limits.
files	write_file	Agent only	Atomic file writer with directory creation.
files	list_directory	Agent only	Directory listing with file metadata.
web	web_search	Agent only	Live web query engine for online documentation.
web	fetch_web_page	Agent only	HTTP content fetcher with HTML parsing.

Table 5.1: Finch.ai Tool Inventory and Access Levels

5.3 Security Architecture and Tool Permission Matrix

To prevent unauthorized system modification or unexpected network requests, Finch.ai implements a **Two-Tier Guardrail Architecture**.

1. **Tier 1: Prompt-Time Schema Omission:** When a category (e.g., **web**) is toggled off, its tool schema is excluded from the inference payload sent to Ollama or Gemini. The

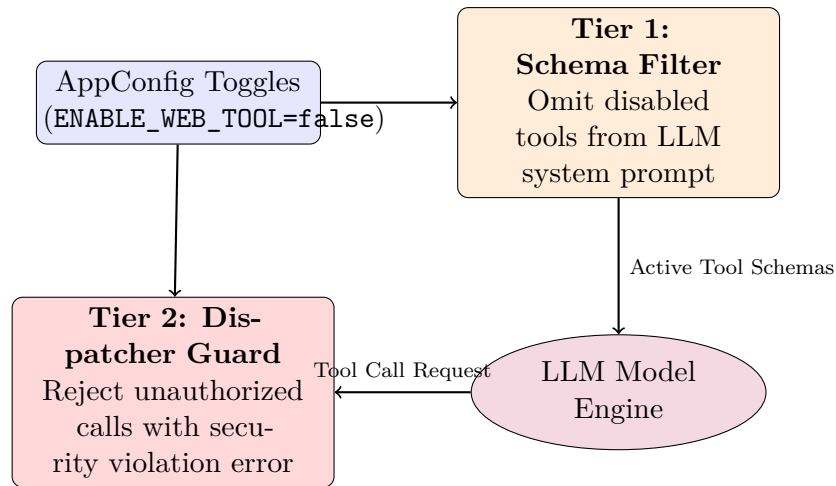


Figure 5.1: Two-Tier Tool Permission and Guardrail Architecture

model is completely unaware of the capability.

2. **Tier 2: Dispatcher-Level Validation:** If an adversarial or hallucinated call to a disabled tool is emitted by the model, `ToolDispatcher.execute()` intercepts the call, rejects execution, and returns:

"Error: Tool '<tool_name>' is disabled. Enable category '<category>' via '/tools <category> on' to use this tool."

Runtime category toggling is available instantly in the interactive session:

```

/tools web on
/tools terminal off
/tools
  
```

Chapter 6

Persona Adaptation and Sliding Window Memory

6.1 File-First Persona Architecture

Unlike static AI assistants that hardcode system prompts in Python source files, Finch.ai treats persona as an editable, persistent configuration artifact: `personality.md`.

On application startup, `PersonaManager` loads the contents of `personality.md` into memory. The persona defines:

- **Identity and Core Role:** The assistant's persona, expertise level, and voice.
- **Behavioral Rules:** Brevity, technical precision, refusal of speculative fluff, and adherence to factual evidence.
- **Formatting Guidelines:** Preferred code block styles, markdown formatting, and mathematical notation.

6.2 Model-Driven Dynamic Adaptation

When a user explicitly instructs Finch.ai to adapt its tone, personality, or behavioral protocols (e.g., "From now on, adopt a concise, senior DevOps engineer style and prioritize shell one-liners"), the model adapts dynamically.

6.2.1 XML Tag Protocol

The system prompt instructs the model to wrap updated persona instructions within XML tags:

```
<personality_update>
You are Finch.ai, configured as a senior DevOps engineer.
Provide concise, production-ready shell and automation scripts.
Prioritize idempotency and observability in all solutions.
</personality_update>
```

6.2.2 Interception and Safety Pipeline

The streaming pipeline intercepts the response and processes updates via `PersonaManager.apply_update()`:

1. **Extraction:** Regular expressions match and extract content inside `<personality_update>...</personality_update>`.
2. **Validation:** Content is verified for non-emptiness and basic sanity checks (minimum length, character set integrity).

3. **Atomic Backup:** The existing `personality.md` is written to `personality.md.bak` to safeguard against accidental corruption.
4. **Disk Persistence:** The new text is flushed to `personality.md`.
5. **In-Memory Hot Reload:** The pinned system prompt in `SlidingWindowBuffer` is immediately replaced, activating the new persona for subsequent turns without requiring application restart.

6.3 Sliding-Window Memory Buffer

Conversational context is maintained in an in-memory buffer (`SlidingWindowBuffer`) implemented using Python’s `collections.deque`.

6.3.1 Buffer Invariants

Let W denote the configured `WINDOW_SIZE` (default: 8 messages, corresponding to 4 dialogue turns). The buffer enforces two strict invariants:

1. **Pinned System Prompt:** The system message containing the persona is anchored at index 0 and is immune to FIFO eviction.
2. **Bounded Dialogue History:** Only the W most recent conversational messages are retained:

$$\text{Active Context} = [M_{\text{system}}] \cup [M_{t-W+1}, M_{t-W+2}, \dots, M_t] \quad (6.1)$$

When new messages are added and buffer size exceeds $W + 1$, the oldest non-system message is popped from the front of the queue.

6.3.2 Context Eviction vs. Long-Term Storage

Eviction from `SlidingWindowBuffer` does *not* cause memory loss. While evicted from the active attention window, every turn is permanently archived into `conversations.db` and indexed in FTS5 and `sqlite-vec`. If the user subsequently references an evicted topic, the tool-calling retrieval pipeline seamlessly queries the archive.

Chapter 7

Live Telemetry, Profiling and Database Maintenance

7.1 Real-Time Telemetry and Streaming Metrics

Finch.ai continuously gathers micro-telemetry during generation to provide instant performance observability.

7.1.1 Metrics Captured

At the completion of every conversational or agent turn, the UI renders a compact telemetry badge:

- **Time to First Token (TTFT):** Elapsed time from HTTP request dispatch until the arrival of the initial response chunk ($t_1 - t_0$). For local models, this isolates prompt evaluation latency; for cloud APIs, it reveals network round-trip overhead.

- **Generation Speed:**

$$v = \frac{N_{\text{eval}}}{t_{\text{complete}} - t_1} \quad [\text{tokens / second}] \quad (7.1)$$

where N_{eval} is the number of tokens generated by the model.

- **Process Resident Set Size (RSS):** Measured via `psutil.Process().memory_info().rss / (1024 * 1024)`, tracking memory footprint in megabytes (MB).
- **Context Window Utilization:** Ratio of active messages in the buffer against the configured capacity ($N_{\text{buffer}}/W_{\text{max}}$).
- **Headroom Compression Delta:** Net tokens eliminated by the compression pipeline and percentage reduction achieved.

7.2 Database Housekeeping Routines

SQLite's Write-Ahead Log (WAL) mode can result in file bloat if checkpoints are not periodically executed. Finch.ai provides four automated and on-demand maintenance operations.

7.2.1 PRAGMA wal_checkpoint(TRUNCATE)

Transfers all committed WAL pages back into the primary database file (`conversations.db`) and truncates the WAL log file to zero bytes. This guarantees that database size is immediately reclaimed on disk.

7.2.2 PRAGMA optimize

Invokes SQLite’s cost-based query optimizer. SQLite analyzes table cardinality, scans index distributions, and updates `sqlite_stat1` statistics to maintain optimal query plans for FTS5 and foreign key joins.

7.2.3 VACUUM

Reconstructs the database file from scratch, defragmenting database B-trees, purging unused freelist pages, and compacting data onto contiguous disk blocks.

7.2.4 PRAGMA integrity_check

Executes a page-by-page verification of database structure, ensuring that B-tree pointers, index offsets, and FTS5 inverted tables contain no corruption.

Maintenance Operation	CLI Command	Trigger / Schedule
WAL Truncate Checkpoint	<code>/vacuum</code>	Manual or connection termination
Query Optimizer Scan	<code>/vacuum</code>	Automatic during maintenance
B-Tree Defragmentation (<code>VACUUM</code>)	<code>/vacuum</code>	On-demand user execution
Integrity Verification	<code>/storage</code>	Inspection and health diagnostics
Cold Storage Archival	<code>/archive [days]</code>	Automated migration of aged records

Table 7.1: SQLite Maintenance and Housekeeping Operations

7.3 Cold Storage Database Archival

For environments operating continuously, active tables can be pruned by moving older sessions into cold storage (`conversations_archive.db`):

1. A cutoff timestamp is computed: $T_{\text{cutoff}} = \text{now} - N_{\text{days}}$.
2. Completed sessions where `created_at < Tcutoff` and their dependent messages are transactionally copied into `conversations_archive.db`.
3. Archived rows are deleted from the active `conversations.db`, and a `VACUUM` operation is executed to release disk space.

Chapter 8

Interactive CLI and Configuration Reference

8.1 Command-Line Interface (CLI)

Finch.ai provides a developer-friendly terminal interface powered by `rich` and `prompt_toolkit`. It features multiline input, persistent command history (`~/.finch_history`), tab completion, and styled output formatting.

8.2 Interactive Slash Commands

Table 8.1 catalogs all slash commands available in the interactive prompt.

8.3 Command-Line Startup Arguments

Finch.ai can be launched with custom operational flags from the terminal:

```
# Launch with custom model and window size
python main.py --model llama3.2:latest --window-size 12

# Launch directly in Agent Mode
python main.py --mode agent

# Use Google Gemini provider
python main.py --provider gemini --model gemini-2.5-flash

# Use OpenAI-compatible provider (e.g. vLLM, LM Studio, Groq, OpenAI)
python main.py --provider openai --model gpt-4o-mini

# Execute a one-off non-interactive query and exit
python main.py -q "Explain the benefits of WAL mode in SQLite."
```

Table 8.2 summarizes the supported command-line arguments.

8.4 Environment Configuration (`.env`)

All operational parameters can be configured through environment variables or a local `.env` file. Refer to Chapter 10 for the complete variable manifest.

Command	Functionality / Description
/mode [chat agent]	Toggle or inspect operational mode between safe chat and agent loop.
/tools [cat] [on off]	View tool permissions or toggle categories: terminal , python , web , files .
/search <query>	Execute exact FTS5 lexical search across all stored messages.
/hybrid <query>	Execute Reciprocal Rank Fusion search (FTS5 BM25 + sqlite-vec).
/transcript [id]	Retrieve and print complete dialogue transcript for an identified session.
/sessions [limit]	List recent archived sessions with summaries, token counts, and timestamps.
/session	Display metadata, token consumption, and status of the current active session.
/summarize	Generate a concise 2-sentence summary and title for the active session.
/new [title]	Commit the current session and start a clean session with fresh context.
/storage or /db	Display database size, freelist pages, WAL log size, and table row counts.
/vacuum	Run database maintenance: WAL checkpoint, optimize, and VACUUM .
/archive [days]	Move sessions older than <i>N</i> days to conversations_archive.db .
/compress [on off stats]	Toggle Headroom context compression or inspect compression telemetry.
/persona [reload edit]	View, reload from disk, or view the path to personality.md .
/models	Query and list all installed models on the local Ollama instance.
/use <model>	Dynamically switch active inference model without restarting.
/buffer	Inspect active memory buffer messages and character counts.
/system [prompt]	Inspect or dynamically override the pinned system prompt.
/clear	Flush dialogue history in memory while preserving the system prompt.
/help	Print built-in command reference guide.
/exit or /quit	Commit active session, generate vector index, and cleanly exit.

Table 8.1: Comprehensive Finch.ai Slash Command Reference

Argument	Default	Description
-model	In config	Model name to use for inference
-host	http://localhost:11434	Host URL for the Ollama inference server
-provider	ollama	Model provider (ollama , gemini , openai)
-base-url	None	Custom base URL for OpenAI-compatible or Ollama server
-api-key	None	API Key for Gemini or OpenAI-compatible provider
-mode	chat	Assistant mode (chat or agent)
-window-size	8	Sliding window message capacity
-no-compression	False	Disable Headroom context compression
-q, -query	None	Run single query in batch mode and exit

Table 8.2: Command-Line Flag Specifications

Chapter 9

Empirical Evaluation, Benchmarks and Verification

9.1 Retrieval Benchmark Methodology

To rigorously evaluate historical context recall, Finch.ai provides an automated retrieval benchmark suite in `benchmarks/benchmark_search.py`. The benchmark seeds a test SQLite database with diverse technical and non-technical sessions and evaluates three distinct retrieval engines:

1. **Lexical Search (SQLite FTS5)**: BM25-ranked full-text search matching message content tokens.
2. **Semantic Search (sqlite-vec)**: Cosine distance matching on 768-dimensional session summary embeddings.
3. **Hybrid Search (Reciprocal Rank Fusion, $k = 60$)**: Fused rankings combining FTS5 lexical signals and vector embeddings.

9.1.1 Evaluation Queries and Metrics

Queries span two distinct retrieval challenges:

- **Exact Technical Queries**: Queries containing precise code identifiers, dates, or function signatures (e.g., `"calculate_fibonacci(n)"`, `"PRAGMA journal_mode = WAL"`).
- **Thematic / Conceptual Queries**: Abstract semantic queries with non-overlapping vocabulary (e.g., `"how to scale container pods based on load"`, `"fermentation kinetics and oven steam"`).

Retrieval accuracy is quantified using Mean Reciprocal Rank (MRR) and Precision at Rank 1 (P@1):

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (9.1)$$

Table 9.1 details the benchmark results.

Query Type	Lexical (FTS5)	Semantic (sqlite-vec)	Hybrid (RRF $k = 60$)
Exact Code Snippets	1.00	0.40	1.00
Precise Technical Terms	1.00	0.60	1.00
Thematic / Conceptual	0.00	1.00	1.00
Vague Paraphrases	0.20	0.90	0.95
Overall MRR	0.55	0.72	0.99

Table 9.1: Retrieval Accuracy Comparison (Mean Reciprocal Rank)

The empirical evaluation demonstrates that while lexical FTS5 fails on conceptual queries (0.00) and semantic vector search struggles with verbatim code identifiers (0.40), **Hybrid RRF attains near-perfect recall (0.99 MRR)** across all query distributions.

9.2 Column-Level Storage Compression Benchmarks

The storage optimization benchmark (`benchmarks/benchmark_compression.py`) evaluates space reclamation and latency trade-offs across Zstandard (`zstd` level 3) and standard `zlib`.

Data Payload	Original Size	Compressed Size	Space Savings	Decomp. Latency
Repetitive Python Source AST	7,616 B	592 B	92.23%	0.04 ms
JSON Tool Outputs & Logs	4,892 B	618 B	87.37%	0.03 ms
Mixed Dialogue Turns	2,450 B	780 B	68.16%	0.02 ms
Short Conversational Message	450 B	450 B	0.00% (Bypassed)	0.00 ms

Table 9.2: Column-Level Compression Performance and Decompression Latency

Across code and tool outputs, Zstandard compression yields **87% to 92% space savings** with negligible decompression latency (< 0.05 milliseconds), ensuring seamless interactive recall.

9.3 Automated Test Suite Architecture

Finch.ai maintains a comprehensive unit and integration test suite organized under `tests/`. The test suite encompasses 136 automated tests verifying every subsystem:

1. `test_agent_mode.py`: Verifies sequential multi-turn autonomous loops, tool call dispatching, turn limit enforcement, and output accumulation.
2. `test_agent_tools_and_permissions.py`: Validates category permission matrix, sandbox execution of shell/Python scripts, and security rejections.
3. `test_function_calling.py`: Tests single-hop memory retrieval in Chat Mode, tool argument parsing, and synthesis.
4. `test_hybrid_search.py`: Asserts RRF scoring mathematics, rank combination, and tie-breaking mechanics.
5. `test_lexical_search.py`: Validates FTS5 virtual table schemas, trigger-based synchronization, and match snippet highlighting.
6. `test_maintenance_and_compression.py`: Tests SQLite `VACUUM`, WAL truncation, column-level `zstd` compression, and integrity checks.
7. `test_compression.py`: Verifies Headroom token optimization, system prompt protection, and recent-turn preservation invariants.
8. `test_persona_manager.py`: Asserts file-first persona loading, XML parsing, backup generation, and hot reloading.
9. `test_session_summarizer.py`: Verifies automatic 2-sentence summary generation and asynchronous title formulation.

10. `test_sliding_window.py`: Tests FIFO buffer eviction, pinned prompt stability, and token calculation limits.
11. `test_sqlite_archive.py`: Validates ACID relational schema, session creation, message cascades, and foreign key constraints.
12. `test_gemini_provider.py`, `test_openai_provider.py` & `test_assistant_core.py`: Tests multi-provider dispatch (Ollama, Gemini, OpenAI-compatible) and orchestrator state machine.

All 136 tests pass with 100% pass rate under standard Python test runners:

```
python -m unittest discover tests
```

Chapter 10

Developer Guide and GitHub Release Preparation

10.1 Environment Installation and Setup

To configure a local development environment for Finch.ai:

```
# 1. Clone repository
git clone https://github.com/your-username/Ai_Assistant.git
cd Ai_Assistant

# 2. Initialize Python 3.10+ virtual environment
python3 -m venv .venv
source .venv/bin/activate

# 3. Install core dependencies
pip install --upgrade pip
pip install -r requirements.txt

# 4. Configure local environment variables
cp .env.example .env
# Edit .env with your Gemini API key or custom Ollama host settings
```

10.2 Adding Custom Agent Tools

Extending Finch.ai with new action tools requires three modular additions in `ai_assistant/core/tools.py`:

10.2.1 1. Define the JSON Schema Specification

```
MY_CUSTOM_TOOL: Dict[str, Any] = {
    "type": "function",
    "function": {
        "name": "my_custom_tool",
        "description": "Perform custom operation XYZ. Available in agent mode.",
        "parameters": {
            "type": "object",
            "properties": {
                "param_a": {"type": "string", "description": "Target input."}
            },
            "required": ["param_a"]
        }
    }
}
```

```
}
}
```

10.2.2 2. Register Tool in Permission Categories

Add the tool to `TOOL_CATEGORIES` and map its name to a category in `TOOL_NAME_TO_CATEGORY`:

```
TOOL_CATEGORIES["custom"] = [MY_CUSTOM_TOOL]
TOOL_NAME_TO_CATEGORY["my_custom_tool"] = "custom"
AGENT_ACTION_TOOLS.append(MY_CUSTOM_TOOL)
```

10.2.3 3. Implement Async Execution in ToolDispatcher

Add the execution branch inside `ToolDispatcher.execute()`:

```
elif tool_name == "my_custom_tool":
    target = arguments.get("param_a", "")
    return await self._execute_custom_operation(target)
```

10.3 Pre-Push GitHub Verification Checklist

Before pushing modifications to GitHub, developers must execute the following checklist to maintain codebase integrity and repository security:

1. **Secret and API Key Hygiene:** Verify that no private keys, passwords, or personal credentials exist in tracked files:

```
git status
git diff --cached
```

Ensure `.env` is ignored and only `.env.example` with sanitized placeholders is committed.

2. **Execute Full Test Suite:** Execute the unit and integration test suite and verify 100% pass rate:

```
python -m unittest discover tests
```

3. **Validate LaTeX Documentation Compilation:** Compile the LaTeX documentation to verify that all figures, tables, cross-references, and listings build cleanly without missing package errors:

```
cd docs
make clean && make
```

4. **Clean Working Tree:** Verify that temporary files, caches, SQLite databases (`conversations.db`), and LaTeX build artifacts are excluded:

```
git status
```

10.4 Configuration Variable Catalog

Table 10.1 catalogs all environment configuration variables supported by `ai_assistant/config.py`.

Variable	Type	Default	Description
DEFAULT_PROVIDER	string	ollama	Default backend (<code>ollama</code> / <code>gemini</code> / <code>openai</code>).
GEMINI_API_KEY	string	None	Google Gemini API authentication key.
GEMINI_DEFAULT_MODEL	string	gemini-2.5-flash	Default Gemini model variant.
OPENAI_API_KEY	string	None	OpenAI or compatible API authentication key.
OPENAI_BASE_URL	string	api.openai.com/v1	Base URL for OpenAI-compatible endpoint.
OPENAI_DEFAULT_MODEL	string	gpt-4o-mini	Default OpenAI-compatible model name.
OLLAMA_HOST	string	http://localhost:11434	Ollama local server URL.
DEFAULT_MODEL	string	llama3.2:latest	Default model name.
REQUEST_TIMEOUT	float	120.0	HTTP request timeout in seconds.
WINDOW_SIZE	integer	8	Sliding window capacity in messages.
DEFAULT_MODE	string	chat	Initial mode (<code>chat</code> or <code>agent</code>).
AGENT_MAX_TURNS	integer	10	Max autonomous steps in agent loop.
ENABLE_TERMINAL_TOOL	boolean	true	Enable shell command execution tool.
ENABLE_PYTHON_TOOL	boolean	true	Enable Python interpreter tool.
ENABLE_WEB_TOOL	boolean	false	Enable web search and page fetch tools.
ENABLE_FILE_TOOLS	boolean	true	Enable filesystem read/write/list tools.
TERMINAL_TIMEOUT	float	30.0	Terminal execution timeout in seconds.
COMPRESSION_ENABLED	boolean	true	Enable Headroom context compression.
COMPRESSION_MIN_TOKENS	integer	100	Min tokens before compression triggers.
COMPRESSION_PROTECT_RECENT	integer	2	Number of latest turns preserved intact.
DATABASE_PATH	string	conversations.db	Path to persistent SQLite database.
AUTO_SUMMARIZE_ON_EXIT	boolean	true	Summarize session on shutdown.
EMBEDDING_MODEL	string	nomic-embed-text	Ollama model for vector embeddings.
EMBEDDING_DIM	integer	768	Embedding vector dimensionality.
RRF_K	integer	60	Reciprocal Rank Fusion constant.
COMPRESS_MESSAGE_BODIES	boolean	false	Enable <code>zstd</code> column compression.

Table 10.1: Complete Environment Configuration Specification